

Java Object-Oriented Programming Lab Manual

Comprehensive Code Analysis, UML Class Diagrams & Execution Workflow

Subject: Object-Oriented Programming with Java

Topics: Inheritance, Method Overriding, Interfaces

System Requirements Specification

HARDWARE & SOFTWARE ENVIRONMENT

All experiments documented in this manual were compiled and executed under the following unified system environment:

Component	Specification / Requirement
Operating System	Windows 10 / 11, macOS, or Linux (Ubuntu 20.04+)
Processor (CPU)	Intel Core i3 / AMD Ryzen 3 or higher (1.5 GHz+)
RAM (Memory)	Minimum 4 GB (8 GB recommended for IDE operation)
Storage	500 MB free space for JDK and source code files
Software Environment	Java Development Kit (JDK) 8.0 or higher (JDK 17/21 LTS recommended)
Text Editor / IDE	VS Code, IntelliJ IDEA, Eclipse, or Command Line Terminal

Program 1: Single Inheritance & Method Overriding

ASSIGN2.JAVA

1. Objective

To demonstrate single-level class inheritance using the extends keyword and achieve dynamic method dispatch via method overriding (@Override).

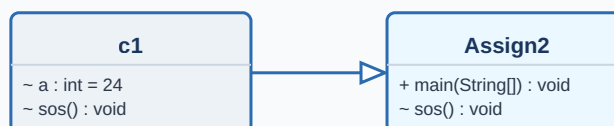
2. Source Code

```
class c1 {
    int a = 24;
    void sos() {
        System.out.println("this is my first class");
    }
}

public class Assign2 extends c1 {
    @Override
    void sos() {
        System.out.println("the value of a is " + a);
        System.out.println("this is my first class");
    }

    public static void main(String[] args) {
        Assign2 ab = new Assign2();
        ab.sos();
    }
}
```

3. UML Class Diagram



4. Algorithm

1. Define superclass c1 with instance variable a = 24 and method sos ().
2. Define public subclass Assign2 extending c1.
3. Override method sos () in Assign2 to print variable a alongside the custom string.
4. In main (), instantiate Assign2 object ab.
5. Invoke ab.sos () to execute overridden method implementation.

5. Compilation & Execution Steps

```
javac Assign2.java  
java Assign2
```

6. Output Screen Terminal

TERMINAL OUTPUT

```
the value of a is 24  
this is my first class
```

Program 2: Hierarchical Inheritance

ASSIGN21.JAVA

1. Objective

To demonstrate hierarchical inheritance in Java where multiple subclasses (bird and sea) extend a single base superclass (animal).

2. Source Code

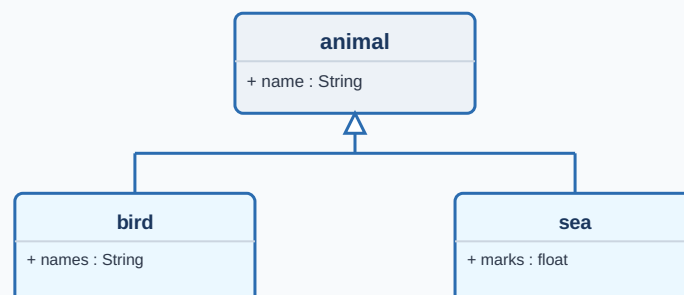
```
class animal {
    String name = " lion";
    void ani() {
        System.out.println(" lion is the my fav animal");
    }
}

class bird extends animal {
    String names = " parrot , sparrow";
    void bi() {
        System.out.println("parrot is my fav bird");
    }
}

class sea extends animal {
    float marks = 99.23f;
    void mar() {
        System.out.println(" you got 1 st rank");
    }
}

public class Assign21 {
    public static void main(String[] args) {
        sea as = new sea();
        as.mar();
        as.ani();
        as.ani();
    }
}
```

3. UML Class Diagram



4. Algorithm

1. Define parent class `animal` containing method `ani()`.
2. Derive subclass `bird` extending `animal`.
3. Derive subclass `sea` extending `animal` containing method `mar()`.
4. In main class `Assign21`, instantiate object as of class `sea`.
5. Invoke `as.mar()` and inherited method `as.ani()`.

5. Compilation & Execution Steps

```
javac Assign21.java  
java Assign21
```

6. Output Screen Terminal

TERMINAL OUTPUT

```
you got 1 st rank  
lion is the my fav animal  
lion is the my fav animal
```

Program 3: Basic Interface Implementation

ASSIGN22.JAVA

1. Objective

To demonstrate interface creation, field implicitly final/static behavior, and concrete implementation using the implements keyword.

2. Source Code

```
interface one {
    int a = 23;
    void two();
}

public class Assign22 implements one {
    public void two() {
        System.out.println("the value of interface is " + a);
    }

    public static void main(String[] args) {
        Assign22 ac = new Assign22();
        ac.two();
    }
}
```

3. UML Class Diagram



4. Algorithm

1. Define interface one containing variable constant a = 23 and abstract method two().
2. Implement interface one in class Assign22.
3. Provide public implementation for method two() displaying variable a.
4. In main(), instantiate object ac and call ac.two().

5. Compilation & Execution Steps

```
javac Assign22.java
java Assign22
```

6. Output Screen Terminal

TERMINAL OUTPUT

```
the value of interface is 23
```

Program 4: Multiple Interface Implementation

ASSIGN23.JAVA

1. Objective

To achieve multiple inheritance in Java by implementing multiple interfaces (first and second) in a single class.

2. Source Code

```
interface first {
    int a = 244;
    void system();
}

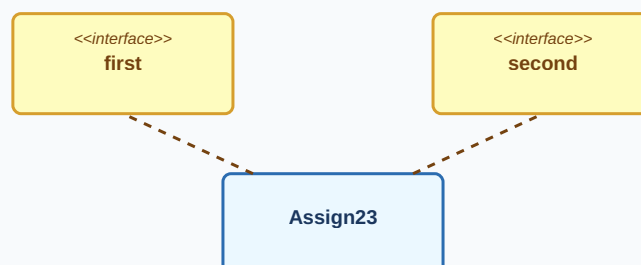
interface second {
    int y = 300;
    void part();
}

public class Assign23 implements first, second {
    @Override
    public void system() {
        System.out.println("the multiple inheritance value is " + a);
    }

    @Override
    public void part() {
        System.out.println("the second interface of value is " + y);
    }

    public static void main(String[] args) {
        Assign23 as = new Assign23();
        as.system();
        as.part();
    }
}
```

3. UML Class Diagram



4. Algorithm

1. Declare interfaces `first` (with variable `a`) and `second` (with variable `y`).
2. Define class `Assign23` implementing both interfaces separated by comma.

3. Implement mandatory abstract methods `system()` and `part()`.
4. In `main()`, instantiate class and invoke both overridden interface methods.

5. Compilation & Execution Steps

```
javac Assign23.java  
java Assign23
```

6. Output Screen Terminal

TERMINAL OUTPUT

```
the multiple inheritance value is 244  
the second interface of value is 300
```

Program 5: Interface Inheritance & Class Implementation

INTERFACE1.JAVA

1. Objective

To demonstrate interface inheritance where interface two extends interface one, and a concrete class implements both.

2. Source Code

```
interface one {
    int a = 244;
    void at();
}

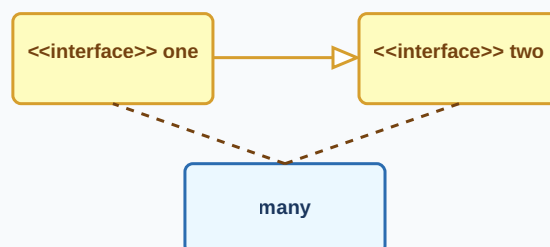
interface two extends one {
    int q = 4;
    void ta();
}

class many implements one, two {
    @Override
    public void at() {
        System.out.println("value is " + a);
    }

    @Override
    public void ta() {
        System.out.println("percent is " + q);
    }
}

public class Interface1 {
    public static void main(String[] args) {
        many m = new many();
        System.out.println(m.a);
        m.at();
        System.out.println();
        m.ta();
    }
}
```

3. UML Class Diagram



4. Algorithm

1. Define interface one with method at ().
2. Define sub-interface two extending one with method ta ().
3. Class many implements one and two, overriding at () and ta ().
4. In driver class Interface1, instantiate many and call methods/fields.

5. Compilation & Execution Steps

```
javac Interface1.java  
java Interface1
```

6. Output Screen Terminal

TERMINAL OUTPUT

244

value is 244

percent is 4

Program 6: Multi-Interface Sub-Interface Hierarchy

INTERFACE2.JAVA

1. Objective

To demonstrate multiple interface inheritance where a child interface d extends multiple parent interfaces (a, b, c).

2. Source Code

```
interface a {
    int x = 44;
    void it();
}

interface b {
    String y = "hemanth";
    void ti();
}

interface c {
    double z = 44.98;
    void ui();
}

interface d extends a, b, c {
    int b = 99;
    void iu();
}

class many implements d {
    @Override
    public void it() { System.out.println(x); }

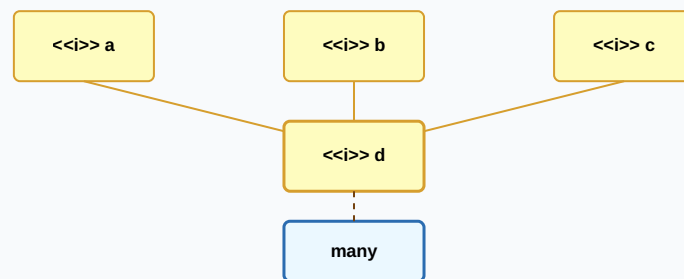
    @Override
    public void ti() { System.out.println(y); }

    @Override
    public void ui() { System.out.println(z); }

    @Override
    public void iu() { System.out.println(b); }
}

public class Interface2 {
    public static void main(String[] args) {
        many n = new many();
        System.out.println(n.y);
        System.out.println(n.x);
        System.out.println(n.z);
        System.out.println(n.b);
        n.it();
        n.iu();
        n.ti();
        n.ui();
    }
}
```

3. UML Class Diagram



4. Algorithm

1. Create interfaces a, b, and c with distinct member variables and methods.
2. Create interface d extending a, b, c simultaneously.
3. Class many implements interface d and fulfills all 4 inherited abstract methods.
4. Main method accesses inherited variables and invokes all implementation methods.

5. Compilation & Execution Steps

```
javac Interface2.java  
java Interface2
```

6. Output Screen Terminal

TERMINAL OUTPUT

```
hemanth  
44  
44.98  
99  
44  
99  
hemanth  
44.98
```

Program 7: Combining Class Inheritance & Interface Implementation

INTERFACE3.JAVA

1. Objective

To demonstrate a class extending a superclass while simultaneously implementing an interface.

2. Source Code

```
class t {
    int y = 44;
    t() {
        System.out.println(y);
    }
    void tv() {
        System.out.println(y + 9);
    }
}

interface v {
    int z = 99;
    void rt();
}

class u extends t implements v {
    int x = 24;

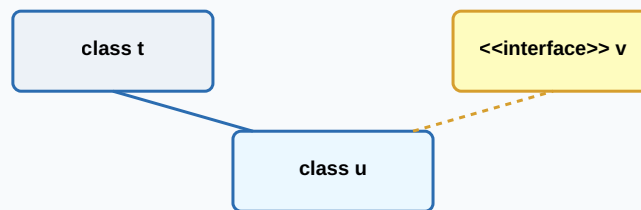
    @Override
    void tv() {
        System.out.println(y + 200);
    }

    u() {
        System.out.println(x);
    }

    public void rt() {
        System.out.println(x + 20);
    }
}

public class Interface3 {
    public static void main(String[] args) {
        u p = new u();
        p.tv();
        p.rt();
    }
}
```

3. UML Class Diagram



4. Algorithm

1. Define superclass t with default constructor printing y (44).
2. Define interface v with abstract method rt ().
3. Subclass u extends t and implements v. Constructor prints x (24).
4. Instantiating u triggers constructor chain execution (t () then u ()).
5. Invoke overridden tv () and interface method rt ().

5. Compilation & Execution Steps

```
javac Interface3.java
java Interface3
```

6. Output Screen Terminal

TERMINAL OUTPUT

```
44
24
244
44
```

Program 8: Hybrid Inheritance with Superclass & Multiple Interfaces

INTERFACE4.JAVA

1. Objective

To demonstrate full hybrid inheritance by deriving a class from a base class while implementing multiple interfaces (extends t implements v, w).

2. Source Code

```
class t {
    int y = 44;
    t() {
        System.out.println(y);
    }
    void tv() {
        System.out.println(y + 9);
    }
}

interface v {
    int z = 99;
    void rt();
}

interface w {
    void ov();
}

class u extends t implements v, w {
    int x = 24;

    @Override
    void tv() {
        System.out.println(y + 200);
    }

    u() {
        System.out.println(x);
    }

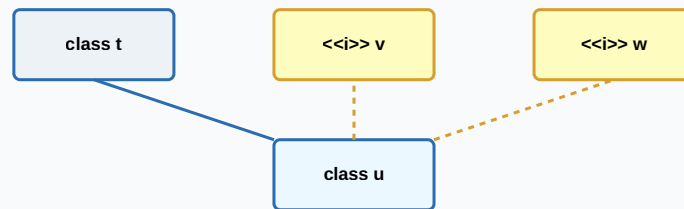
    public void rt() {
        System.out.println(x + 20);
    }

    public void ov() {
        System.out.println("multiple inheritance is completed");
    }
}

public class Interface4 {
    public static void main(String[] args) {
        u o = new u();
        o.ov();
        o.rt();
        o.tv();
    }
}
```

```
}  
}
```

3. UML Class Diagram



4. Algorithm

1. Define superclass t and interfaces v and w.
2. Define class u extending class t and implementing both interfaces v , w.
3. Provide implementations for rt () from v and ov () from w, plus overridden tv ().
4. Instantiate object o of class u in main driver method.
5. Execute methods ov (), rt (), and tv ().

5. Compilation & Execution Steps

```
javac Interface4.java  
java Interface4
```

6. Output Screen Terminal

TERMINAL OUTPUT

```
44  
24  
multiple inheritance is completed  
44  
244
```